Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 06

Student ID	IT24102555
Student Name	Weerathunga B.A
Campus/ Center name	SLIIT Malabe
Specialization	Data Science
Batch No	Y2.S2.WD.DS.0102

Activity 1

a.

```
GNU nano 7.2                    lab6activity1.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
  int fd[2];
  pid_t pid;
  char write_msg[] = "Hello!";
  char read_msg[100];

  pipe(fd);
  pid= fork();

  if (pid>0) {
   close(fd[0]);
   write(fd[1], write_msg, strlen(write_msg) +1);
   close(fd[1]);
   wait(NULL);
 }else {
   close(fd[1]);
   read(fd[0], read_msg, sizeof(read_msg));
   printf("Child received: %s\n", read_msg);
   close(fd[0]);
 }
 return 0;
}
```

```
$ nano lab6activity1.c
$ gcc -olab6activity1 lab6activity1.c
$ ./lab6activity1
Child received: Hello!
$
```

b. **Line A-** Creates the pipe. It establishes a unidirectional communication channel between parent and child.

Line B- Creates new process. Both parent and child share the pipe file descriptions.

Line C- Used by parent to write data into the pipe. Data flows from fd[1]

Line D- Used by child to read data from the pipe. Stores it in a buffer for printing.

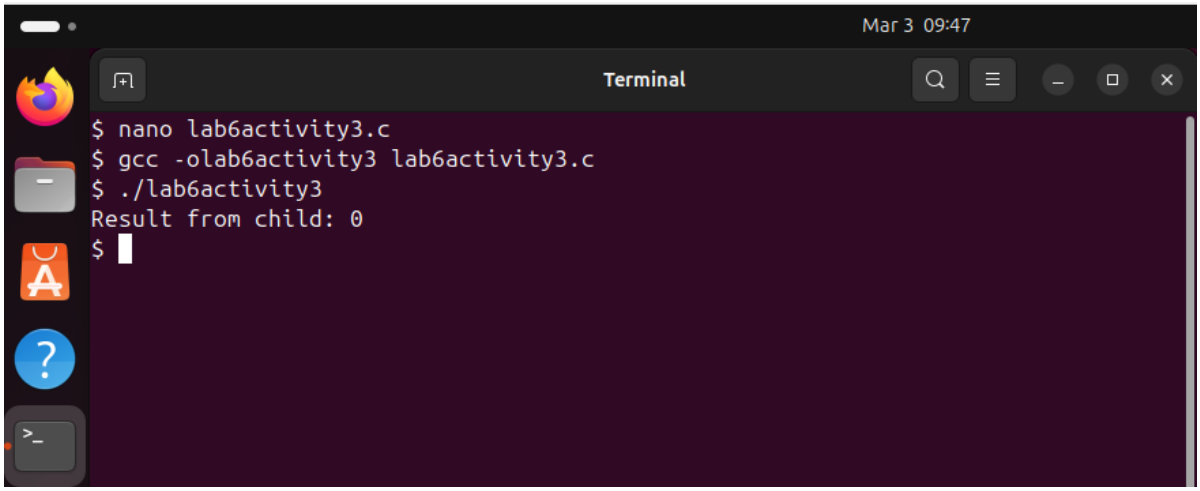
Activity 2

```
Terminal
  GNU nano 7.2                    lab6activity2.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>

int main() {
  int fd[2];
  pid_t pid;
  char message[] = "Hello Parent, this is child!";
  char buffer[100];

  pipe(fd);
  pid= fork();

  if (pid>0) {
   close(fd[1]);
   read(fd[0], buffer, sizeof(buffer));
   printf("Parent recieved: %s\n", buffer);
   close(fd[0]);
 }else {
   close(fd[0]);
   write(fd[1], message, strlen(message)+1);
   close(fd[1]);
 }
 return 0;
}

                                  [ Read 27 lines ]
```

```
$ nano lab6activity2.c
$ gcc -olab6activity2 lab6activity2.c
$ ./lab6activity2
Parent recieved: Hello Parent, this is child!
$ nano lab6activity2.c
$
```

Activity 3

```
GNU nano 7.2                    lab6activity3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
  int pipe1[2];
  int pipe2[2];
  pid_t pid;
  int number, result;

  pipe(pipe1);
  pipe(pipe2);

  if(pid>0) {
    printf("Enter a number: ");
    scanf("%d", &number);

    close(pipe1[0]);
    write(pipe1[1], &number, sizeof(number));
    close(pipe1[1]);
  }
  else {
    close(pipe2[1]);
    read(pipe2[0], &result, sizeof(result));
    printf("Result from child: %d\n", result);
    close(pipe2[0]);

    result= number * 2;


    close(pipe2[0]);
    write(pipe2[1], &result, sizeof(result));
    close(pipe2[1]);
  }
  return 0;
}
```


Mar 3 09:47
Terminal
$ nano lab6activity3.c
$ gcc -olab6activity3 lab6activity3.c
$ ./lab6activity3
Result from child: 0
$